

Ride Sharing System: Object-Oriented Programming in C++ and Smalltalk

Amjad Alqahtani

Computer Science, University of Cumberlands

MSCS-632: Advanced Programming Languages

Dr. Jay Thom

Oct 2, 2026

Ride Sharing System: Object-Oriented Programming in C++ and Smalltalk

Introduction

The purpose of this project was to develop a Ride Sharing System using two object-oriented programming languages, C++ and Smalltalk. The system includes different types of rides, riders, and drivers. The main objectives are demonstrating three important object-oriented programming principles: encapsulation, inheritance, and polymorphism. Although the two languages use different syntax and programming styles, both implementations represent the same application design.

C++ Implementation

The C++ implementation contains a base class named `Ride` and two derived classes named `StandardRide` and `PremiumRide`. The `Ride` class stores common information including the ride ID, pickup location, drop-off location, and distance. It also defines the `fare()` and `rideDetails()` methods.

Encapsulation defined as declaring important data members as private. For example, the `Driver` class contains a private `assignedRides` collection. Rides cannot be added directly from outside the class and must instead be added through the `addRide()` method. Similarly, the `Rider` class keeps the `requestedRides` collection private and provides the `requestRide()` and `viewRides()` methods to interact with the collection.

Inheritance defined as `StandardRide` and `PremiumRide` classes. Both classes inherit common attributes and behaviors from the `Ride` base class. This avoids duplicating common ride information in each subclass. Each subclass then provides its own implementation of the `fare()` method.

Polymorphism is defined as storing both `StandardRide` and `PremiumRide` objects in a collection of `Ride` pointers. The program loops through the collection and calls `fare()` and `rideDetails()` using the base-class pointer. At runtime, C++ selects the appropriate overridden method based on the actual object. Therefore, a `StandardRide` executes the standard fare calculation while a `PremiumRide` executes the premium fare calculation.

Smalltalk Implementation

The Smalltalk implementation follows the same object-oriented design. A general Ride class contains the common ride information, while StandardRide and PremiumRide inherit from Ride. Each subclass implements its own fare calculation.

Encapsulation defined as storing object state in instance variables and accessing or modifying that state through methods. Driver and rider ride collections are managed through methods rather than being manipulated directly. Inheritance allows StandardRide and PremiumRide to reuse the behavior defined in the Ride superclass. The subclasses only need to define behavior that is different, such as the fare calculation. Polymorphism is defined as placing different ride objects inside the same Smalltalk collection and sending the same messages, such as fare and rideDetails, to each object.

Comparison

Both C++ and Smalltalk support encapsulation, inheritance, and polymorphism, but they implement these concepts differently. C++ uses explicit access modifiers such as private and relies on virtual functions and base-class pointers for runtime polymorphism. Smalltalk relies heavily on message passing and dynamic method lookup. Smalltalk is defined polymorphism naturally because the same message can be sent to objects of different classes without requiring explicit pointer types. The C++ implementation provides compile-time type checking, while Smalltalk provides a dynamic one.

Conclusion

Ride Sharing System can be implemented using two different object-oriented languages. Encapsulation protects the internal state of objects, inheritance allows common functionality to be reused, and polymorphism allows multiple ride types to respond differently to the same method call.

Smalltalk output

```
xxx@salemmac RideSharing % ./ride.sh
=== ALL RIDES ===
Type      Ride ID  Pickup      Dropoff      Distance  Fare
-----
Standard   101      University  Airport      10        $20
Premium    102      Mall        Downtown     8         $32

=== POLYMORPHIC FARE TEST ===
Ride ID    Fare
-----
101        $20
102        $32

=== DRIVER INFORMATION ===
Driver ID   Name      Rating
-----
1           John Smith 4.9

Assigned Rides:
Type      Ride ID  Pickup      Dropoff      Distance  Fare
-----
Standard   101      University  Airport      10        $20
Premium    102      Mall        Downtown     8         $32

=== RIDER INFORMATION ===
Rider ID    Name
-----
201         Alice Johnson

Ride History:
Type      Ride ID  Pickup      Dropoff      Distance  Fare
-----
Standard   101      University  Airport      10        $20
Premium    102      Mall        Downtown     8         $32
xxx@salemmac RideSharing %
```

C++ Output

```
5  Compiling Ride Sharing System
22 Compilation successful.
29 Running program...
=====

=== ALL RIDES ===
Type      Ride ID  Pickup      Dropoff      Distance  Fare
-----
Standard   101      University  Airport      10        $20
Premium    102      Mall        Downtown     8         $32

=== Driver Information ===
Driver ID   Name      Rating
-----
1           John Smith 4.9

Assigned Rides:
Type      Ride ID  Pickup      Dropoff      Distance  Fare
-----
Standard   101      University  Airport      10        $20
Premium    102      Mall        Downtown     8         $32

=== Rider Information ===
Rider ID    Name
-----
201         Alice Johnson

Ride History:
Type      Ride ID  Pickup      Dropoff      Distance  Fare
-----
Standard   101      University  Airport      10        $20
Premium    102      Mall        Downtown     8         $32
xxx@salemmac week6 %
```

GitHub Repository: <https://github.com/AmjadAlqahtani90/Ride-Sharing-System>